

H D M L 2 . 0 L A N G U A G E R E F E R E N C E

V e r s i o n 2 . 0

UNWIRED PLANET, INCORPORATED

390 Bridge Parkway
Redwood Shores, California 94065
USA

Part Number HDMLREF-DOC-200
Revision D, July 1997



IMPORTANT NOTICE

COPYRIGHT INFORMATION

Copyright 1997 Unwired Planet, Inc. All rights reserved.

These files are part of the Unwired Planet Software Developer's Kit (the "UP.SDK").

Subject to the terms and conditions of the UP.SDK License Agreement, Unwired Planet, Inc. hereby grants you authorization to use UP.SDK software and its related documentation.

TO THE MAXIMUM EXTENT PERMITTED BY LAW, UNWIRED PLANET MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, REGARDING THE UP.SDK SOFTWARE, INCLUDING, BUT NOT LIMITED TO, ANY IMPLIED WARRANTIES OR MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NONINFRINGEMENT. USER UNDERSTANDS AND ACCEPTS THE UP.SDK SOFTWARE ON AN "AS IS" BASIS FROM UNWIRED PLANET, AND UNWIRED PLANET DOES NOT WARRANT, GUARANTEE, OR MAKE ANY REPRESENTATIONS REGARDING THE USE OF, OR THE RESULTS OF THE USE OF THIS UP.SDK SOFTWARE IN TERMS OF ITS CORRECTNESS, ACCURACY, RELIABILITY, OR OTHERWISE. IN NO EVENT SHALL UNWIRED PLANET BE LIABLE FOR ANY DAMAGES RESULTING FROM OR ARISING OUT OF YOUR USE OF THE UP.SDK SOFTWARE, INCLUDING, WITHOUT LIMITATION, ANY DIRECT, INDIRECT, CONSEQUENTIAL, OR PUNITIVE DAMAGES RESULTING FROM THE USE, MODIFICATION, OR DISTRIBUTION OF THE UP.SDK SOFTWARE OR ITS DERIVATIVES.

Unwired Planet, the Unwired Planet logo, UP.Phone, UP.Link, UP.Browser, UP.Access, UP.Mail, UP.PIM, UP.Organizer, UP.Pager, HDML and UP.SDK are either trademarks or registered trademarks of Unwired Planet, Inc. in the United States.

Contents

Preface v

The UP.SDK vi

Roadmap to UP.Link developer documentation vi

Style and typographical conventions vii

1 Introduction to HDML Structure and Syntax 9

Overview 9

 The abstract phone 9

 The HDML character set 10

Cards and decks 11

 Example 11

Options and option lists 13

Actions 13

 Destinations 14

 A simple example 14

 Action scope and precedence 15

Choice cards and tasks 16

Card history 16

 History pruning 17

Activities 17

 Nested activities 18

 Activity history 19

 Canceling an activity 19

Variables 19

 Variable naming 20

 Variable scope 20

 Setting variables 20

 Capturing user input to variables 21

 Referencing variables 21

 Passing variables between activities 23

Formatted text 23

 Reserved characters 24

 Line breaking, wrapping, and horizontal scrolling 24

 Vertical scrolling 25

 Tabs 25

 Spaces 25

2 HDML Statement Reference 27

- Actions (<ACTION>) 27
 - Action precedence for function keys 27
 - Syntax 29
- Choice cards (<CHOICE>) 33
 - Syntax 33
- Comments 37
 - Syntax 37
- Decks (<HDML>) 37
 - Syntax 38
- Destinations (DEST=) 40
 - Syntax 40
 - Destination examples 41
- Display cards (<DISPLAY>) 41
 - Syntax 41
- Entry cards (<ENTRY>) 43
 - Syntax 43
- Images () 46
- No-display cards (<NODISPLAY>) 47

3 HDML Quick Reference 49

- Defining cards and decks 49
- Displaying formatted text 50
- Specifying actions and tasks 51
- Specifying destinations 52
- Defining images 52

Index 53

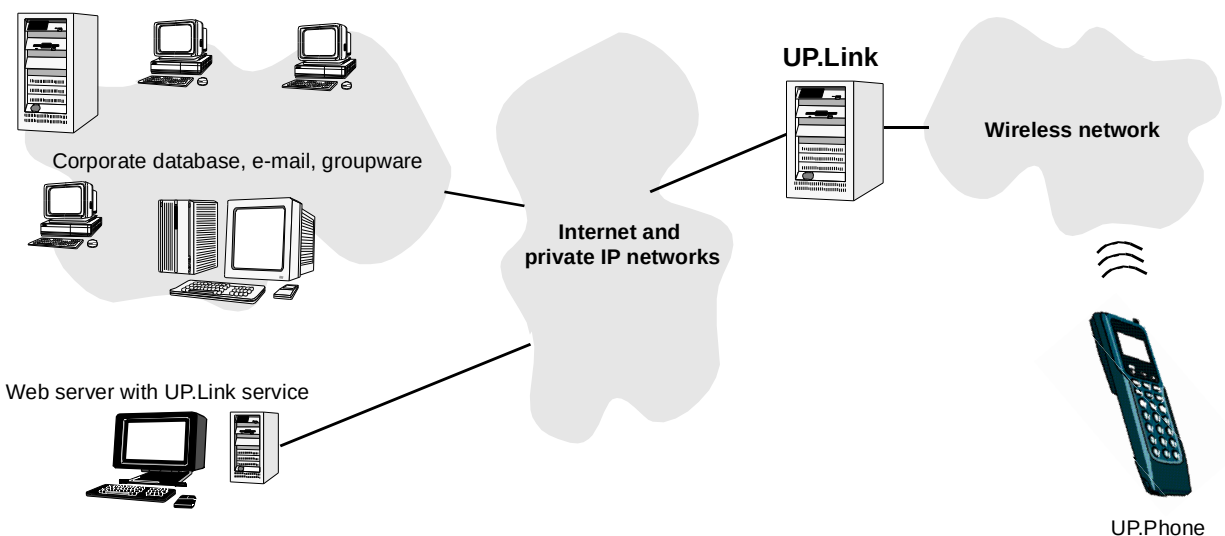
Preface

This manual provides reference information for developers who want to use Unwired Planet SDK (UP.SDK) to create services for the UP.Link™ platform.

The UP.Link platform provides owners of data-capable handheld devices, such as cellular phones, with wireless access to a wide array of services. These services include, but are not limited to, many of the services available on the Internet.

An UP.Link-enabled handheld device, or *UP.Phone*, connects to World Wide Web (WWW) servers through a system called an *UP.Link*. The UP.Phone functions much like a WWW browser; the user presses keys on the phone to navigate and request URLs. The UP.Phone uses the data capabilities of conventional cellular networks to send the requests to an UP.Link, which converts them into Hypertext Transport Protocol (HTTP) requests.

The UP.Link sends HTTP requests over the Internet or direct lines to web servers maintained by Unwired Planet or third-party developers. The applications on these web servers that handle UP.Link requests are called *UP.Link services*. UP.Link services respond to the requests using Handheld Device Markup Language (HDML), an open language developed by Unwired Planet to interactively display information on small handheld devices. When an UP.Link receives a response from an UP.Link service, it relays it back to the UP.Phone that originated the request. The UP.Phone displays the response to the user.



UP.Link services can also initiate communication with a user, by “pushing” alerts and information to an UP.Link, which pushes them to the user’s UP.Phone. This process is called *notification*.

The UP.SDK

The UP.SDK provides tools for developers who want to create or maintain UP.Link services. It includes the following components:

- The UP.Phone simulator for Windows 95 and Windows NT, which simulates the behavior of an UP.Link-enabled device and makes it easy to test UP.Link services
- Libraries of Perl and C functions, which simplify the process of generating HDML and handling HTTP requests
- Sample HDML files and source code, which you can use as a starting point for your own UP.Link services
- UP.Link developer documentation, which consists of online manuals provided in Adobe Acrobat Portable Document Format (PDF)

Roadmap to UP.Link developer documentation

UP.SDK documentation assumes that you are familiar with the Common Gateway Interface (CGI) protocol. To get the most from UP.SDK documentation, you should also be familiar with the HyperText Markup Language (HTML) and programming languages, such as C and Perl.

UP.SDK documentation includes the following manuals:

- The *HDML 2.0 Language Reference*, the manual you are reading now, provides reference information about the HDML language.
- *Getting Started with the UP.SDK* provides instructions for getting started with the UP.SDK. It discusses the basics of generating HDML and providing an UP.Link service. It is recommended that you read this manual first.
- The *UP.SDK Developer’s Guide* describes in detail how to implement an UP.Link service. It also describes how to use sophisticated UP.SDK features, such as faxing and notifications.
- The *UP.SDK Tools and APIs Reference* provides instructions and reference information for tools and APIs provided with the UP.SDK. It includes instructions for using the UP.Phone simulator, as well as reference information on Perl and C libraries that perform operations such as create digests and generate HDML.
- The *UP.SDK 2.0 Transition Guide* provides instructions and background information for developers who want to convert their UP.Link 1.0 services to work with UP.Link 2.0.
- *Using the UP.Browser* provides instructions for using an UP.Phone. Although it is an end user manual, it is useful for getting acquainted with the UP.Phone interface. In particular, it helps you use the UP.Phone simulator.

Style and typographical conventions

UP.SDK documentation distinguishes between *you*, the developer, and *the user* or *the subscriber*, the individuals who use the services you create.

The term *UP.Phone* or *phone* refers to all UP.Link-enabled devices, such as cellular phones, Personal Digital Assistants (PDAs), and two-way pagers. It includes both the device hardware and the UP.Link Browser client software from Unwired Planet that is installed on it.

UP.SDK documentation and the HDML language use the names ACCEPT, PREV, and SOFT1 to specify function keys on the UP.Phone. The actual labels and locations of these keys vary from one device to another. For more information on handheld device configurations, see “UP.Link-enabled devices” in Chapter 1 of the *UP.SDK Developer’s Guide* and the Unwired Planet developer’s web site at:

<http://www.uplanet.com>.

The phone display examples in this manual depict a 4 X 20 character display on a device that has three function keys. Output will appear differently on a device that has a different display size or a different number of function keys. HDML is designed to work with phone displays as small as 3 X 12.

Omitted code in examples is indicated with ellipses. For example, the ellipses in the following HDML code example indicate that some of the code necessary for a complete HDML deck has been omitted:

```
<HDML VERSION=2.0>
. . .
</HDML>
```

Some code listings in this manual have line numbers, which appear to the left of the code. For example:

```
1 <HDML VERSION=2.0>
2   <DISPLAY>
3     A deck.
4   </DISPLAY>
5 </HDML>
```

These line numbers are used for reference purposes only and are not part of the actual code.

This manual uses different fonts to represent different types of information.

- Text that you enter, function names, filepaths, and Uniform Resource Locators (URLs) appear in `text like this`.
- Required HDML keywords and characters in syntax descriptions appear in **text like this**.
- Optional HDML keywords and characters in syntax descriptions appear in *text like this*.
- Placeholders that you replace with your own text appear in *text like this*.

Preface

Style and typographical conventions

Introduction to HDML

Structure and Syntax

1

This chapter provides a high-level description of the structure and syntax of the HDML language. This chapter is intended only as an introduction to HDML structure and syntax; for a complete description of HDML statements and options, see Chapter 2, “HDML Statement Reference.”

Overview

Although the syntax of HDML is similar to that of Hypertext Markup Language (HTML), HDML is not a true markup language. It is a set of commands or statements that specify how a handheld device, such as a cellular phone, interacts with a user.

HDML statements display information on a phone and specify how the phone responds to user input. For example, an HDML statement can instruct a phone to display text and, based on the user’s input, either display additional text or send a request to the UP.Link.

The abstract phone

Because HDML can be used on a variety of handheld devices, with significantly different capabilities, it is defined with respect to a reference, or abstract, phone. The abstract phone has the following minimum feature set:

- 12-fixed width character display
- Vertical scrolling
- ASCII printable character set
- Message waiting indicator
- Numeric and alphabetic character entry
- Data entry editing
- Choice selection (using arrow or numeric keys)
- ACCEPT and PREV keys
- One or two programmable keys (*softkeys*) supporting labels of five characters or more

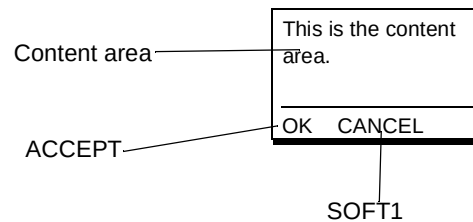
- Storage of state, including the current deck, card, choice, activity stack, variables, and destination card.

The phone can also support optional features such as variable-width characters, ISO Latin-1, and Unicode character sets; bold and italic characters; and bitmap graphic display.

Many examples in this manual and other UP.SDK documentation include phone displays similar to the one in Figure 1-1. In these displays, the ACCEPT key label always appears in the lower left-hand corner and the first soft key (SOFT1) label appears at the bottom-center of the display.

The PREV key and non-programmable keys (such as the arrow keys and the numeric keys) are not shown in the examples, because their labels are not programmable; they appear on the keys themselves and not the phone display.

FIGURE 1-1. The ACCEPT and SOFT1 softkey labels



The HDML character set

The HDML character set includes all printable 8-bit characters and *white space*. In HDML, white space is defined as the following characters.

Character	Decimal 8-bit value
Newline	10
Carriage return	13
Space	32
Tab	9

Except where specifically noted, the HDML compiler ignores white space beyond a single character. For example, the following HDML code:

```
<HDML VERSION=2.0>< DISPLAY> <WRAP>Some          text
</DISPLAY >

</HDML>
```

is equivalent to:

```
<HDML VERSION=2.0>
  <DISPLAY>
    <WRAP>Some text
  </DISPLAY>
</HDML>
```

The HDML compiler and the UP.Phone ignore case in HDML keywords. By convention, UP.SDK manuals and sample code capitalize HDML keywords, such as tags and options, so you can distinguish them from variable names and other text that you provide. However, this is only a style convention, which you can ignore when you write your own HDML.

IMPORTANT Case is important in names that you provide in HDML, such as card and variable names. For example, the variable names `mycard` and `Mycard` are different.

Cards and decks

The smallest unit of HDML an UP.Link service can send to a phone is a *deck*¹. A deck contains one or more cards, each of which specifies a single interaction between the phone and the user. HDML currently supports the following types of cards:

- *Entry cards*—which display a message and allow the user to enter a string of text
- *Choice cards*—which display a list of options from which the user can choose a single option
- *Display cards*—which simply display information
- *No-Display cards*—which execute an action but do not appear on the phone display

When a phone receives a deck, it normally goes to the first card in the deck. It displays the information the card provides and allows the user to respond to it. Depending on the card type, the user can respond by entering text or choosing an option, and then pressing a function key.

Example

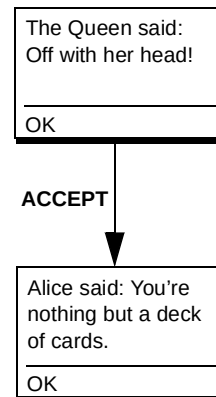
The following HDML statements are an example of an HDML deck containing two display cards. When a phone loads this deck, its display appears similar to the display at the top of Figure 1-2. If the user presses ACCEPT (the softkey below the OK label),

1. The UP.Link and the UP.Phone support several content types besides HDML decks. A service can use one of these types, the UP.Link digest, to send multiple HDML decks to an UP.Phone in a single transmission. UP.Link digests and other content types are beyond the scope of this manual. For more information about them, see the *UP.SDK Developer's Guide*.

the phone display appears similar to the display at the bottom of Figure 1-2. Following Figure 1-2 is a line-by-line description of the HDML.

```
1 <HDML VERSION=2.0>
2   <DISPLAY>
3     <ACTION TYPE=ACCEPT TASK=GO DEST=#card2>
4     The Queen said: Off with her head!
5   </DISPLAY>
6   <DISPLAY NAME=card2>
7     Alice said: You're nothing but a deck of cards.
8   </DISPLAY>
9 </HDML>
```

FIGURE 1-2. HDML deck with two cards



Line 1

This line defines the header of the HDML deck. All HDML decks must begin with an `<HDML>` tag and end with an `</HDML>` tag.

Most HDML tags, such as the `<HDML>` tag, allow you to specify *options*. Options take the form, `OPTION=value`, where *OPTION* is the option name and *VALUE* is an alphabetic or numeric value that you specify. Some options, such as the `VERSION` option in the HDML tag, are required. Option syntax is described in more detail in the following section.

Line 2

This line defines the header of the first display card. Like HDML decks, cards require begin and end tags (for example, `<DISPLAY>` and `</DISPLAY>`).

Line 3

This line defines an *action*, which specifies what the phone should do when the user presses a specified function key. The `TYPE` option identifies the key (`ACCEPT`), the `TASK` option identifies what the phone should do (go to another deck or card), and the `DEST` (destination) option identifies where to go (the card named `card2`).

Line 4

This line provides the text that is displayed on the phone.

Line 6

This line defines the header of the second display card. It uses the `NAME` option to specify the card's name. Although the `NAME` option is not required, it is necessary to specify a name so that other cards can reference the card as a destination.

Options and option lists

Many HDML statements allow you to specify options. Individual options have the following syntax:

option=value

option is a string of alphanumeric characters for which case is ignored. *value* can contain alphanumeric characters or nonalphanumeric characters. If the value contains white space you must enclose it in double quotation marks ("). If it contains reserved HDML characters (such as `<`, `>`, or `$`), you must escape these characters, using URL escaping conventions. If the value contains only alphanumeric characters, quotation marks are optional. HDML syntax does not allow white space between the key, the equal sign, and the value.

Many HDML statements allow you to specify an *option list*, or a concatenation of individual options separated by white space. The syntax for an option list is:

option1=value1 option2=value2 option3=value3 . . .

Actions

HDML allows you to define *actions*, which associate a phone function key with a *task*, such as downloading and displaying a specified card or deck, or dialing a phone number.

If you do not define an action for a key, the phone executes the key's default task when the user presses it. The following table lists supported function keys and their default tasks:

Key	Default task
ACCEPT	Display the previous card
HELP	Display a built-in card containing a message that no help is available for the current location.
PREV	Display the previous card or cancel the activity if there is no previous card
SOFT1-SOFT2	Do nothing

Destinations

Destinations are URLs that tasks use to specify decks and cards to download and display on the phone. Destinations are similar to conventional World Wide Web URLs, with several extensions:

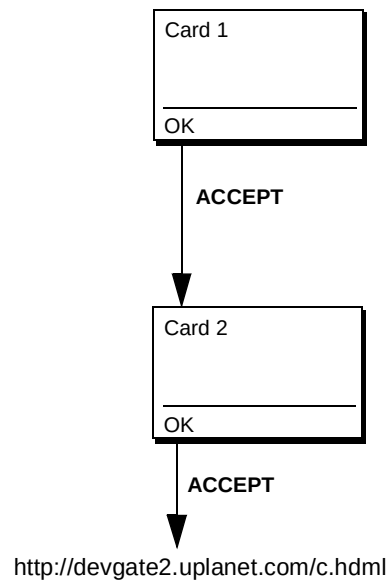
- A destination that identifies an HDML deck uses a conventional URL, for example:
`http://devgate2.uplanet.com/examples/password.html`
- The first card in a deck has the same URL as the deck itself.
To identify another card in the deck, you append a hashmark (#) and the cardname to the deck URL. For example, the following destination identifies a card named `card2` in the `password.html` deck:
`http://devgate2.uplanet.com/examples/password.html#card2`
- If you are referencing a card that is within the same deck, you can use the card name by itself, for example:
`#card2`

A simple example

The following HDML deck contains two cards, which specify actions. The action in the first card instructs the phone to display the second card when the user presses `ACCEPT`; the action in the second card instructs the phone to request the URL `http://devgate2.uplanet.com/` when the user presses `ACCEPT`.

```
<HDML VERSION=2.0>
  <DISPLAY NAME=card1>
    <ACTION TYPE=ACCEPT TASK=GO DEST=#card2>
      Card 1
    </DISPLAY>
  <DISPLAY NAME=card2>
    <ACTION TYPE=ACCEPT TASK=GOSUB
      DEST= http://devgate2.uplanet.com/c.html>
      Card 2
    </DISPLAY>
</HDML>
```

FIGURE 1-3. Cards that specify actions



Action scope and precedence

You can define actions at both the deck and the card level. The scope of an action defined at the deck level is the entire deck. The scope of an action defined at the card level is the current card only. Actions defined at the card level take precedence over actions defined at the deck level.

For example, the following deck defines actions at both the deck and the card level.

```
<HDML VERSION=2.0>
  <ACTION TYPE=ACCEPT TASK=GO DEST=#cardA>
  <ACTION TYPE=SOFT1 TASK=GO DEST=#cardC>
  <DISPLAY NAME=card1>
    <ACTION TYPE=ACCEPT TASK=GO DEST=#cardB>
    Card 1
  </DISPLAY>
  . . .
</HDML>
```

If the user presses SOFT1 while the phone is displaying card1, the phone goes to card. However, if the user presses ACCEPT, the phone goes to cardB—that is, it executes the ACCEPT action defined by the Display card and ignores the ACCEPT action defined by the deck.

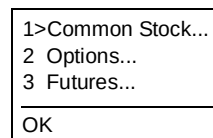
Choice cards and tasks

The previous section describes how you use an action to assign a task to a function key. HDML also allows you to assign tasks to individual choices in a choice card. For example, the following deck provides three choices.

```
<HDML VERSION=2.0>
  <ACTION TYPE=ACCEPT TASK=GO DEST=#other_card>
  <CHOICE>
    <CE TASK=GO DEST=#stock>Common Stock...
    <CE TASK=GO DEST=#options>Options...
    <CE TASK=GO DEST=#futures>Futures...
  </CHOICE>
  . . .
</HDML>
```

When the user chooses one of these choices and presses OK, the phone goes to the corresponding destination. Note that the phone ignores the ACCEPT action defined by the deck. Tasks defined for individual choices take precedence over deck and card-defined actions.

FIGURE 1-4. Choice card



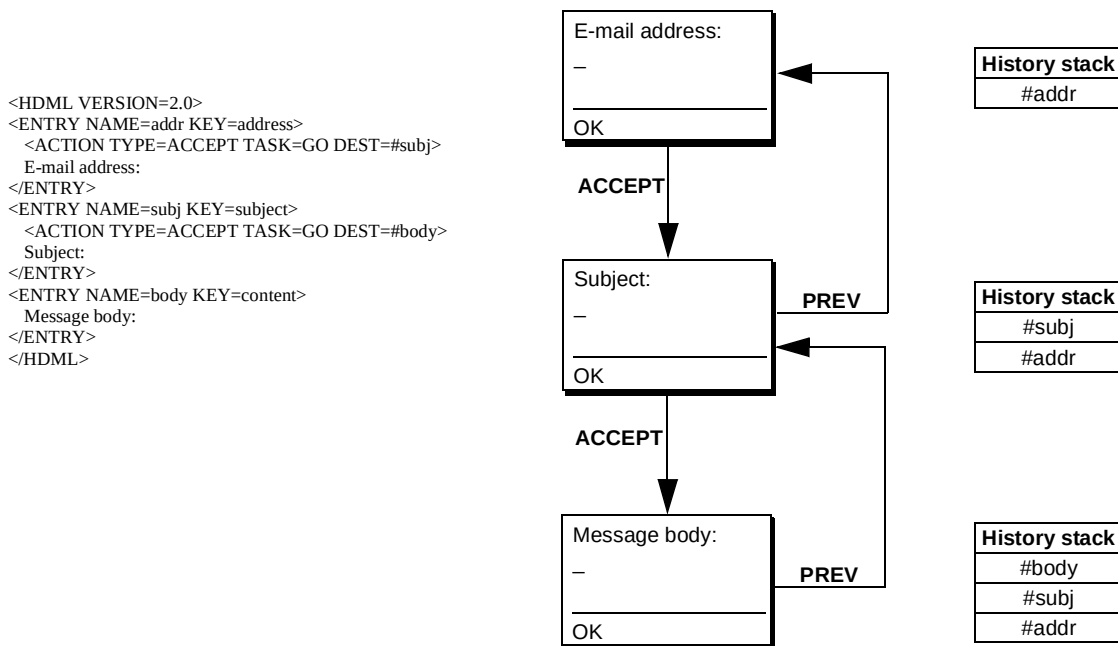
Card history

The phone maintains a history stack of the cards a user visits, enabling the user to navigate backward through the cards by pressing PREV. Each time the user navigates forward to a card, the phone pushes the card onto the history stack. Each time the user presses PREV to navigate backward, the phone pops the current card off the stack, leaving the previous card at the top of the stack.

The UP.Phone history model differs slightly from the model used by most WWW browsers. Most WWW browsers provide Back and Forward commands that allow you to navigate backward *and* forward through the history. On a UP.Phone, the current card is always at the top of the history. You can only navigate backward through the history.

For example, suppose an email service provides sequential cards to enter an email message address, subject, and body. After entering the address and the subject, the user can back up to either card by pressing PREV. Figure 1-5 illustrates this interface. The HDML for the cards is shown on the left; the state of the history stack at each point in the navigation is shown on the right.

FIGURE 1-5. Card history



History pruning

When the user presses the Home key on the phone, the phone displays the user's home card. This trims (prunes) the history stack, so that the home card is the only card in the stack. If you override the default action for the PREV key, it also prunes the history stack to the current card.

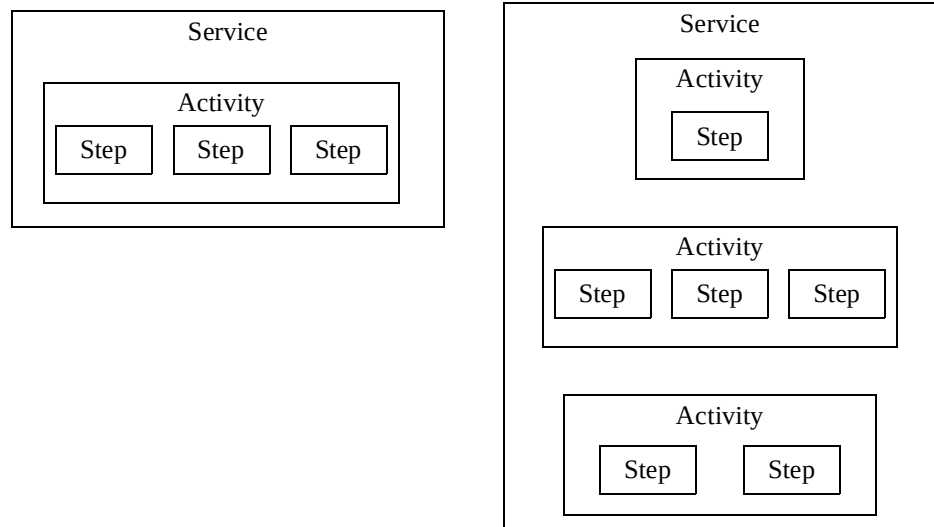
Activities

HDML applications are structured around *activities*. Activities correspond to tasks the user wants to carry out. Each activity consists of one or more steps.

An HDML service can provide multiple activities to its users. For example, a mail service might provide activities such as "Create a New Message" and "View Mail Inbox". The "Create a New Message" activity might include steps to enter an e-mail address, enter a subject, and enter the message body.

Figure 1-6 provides a conceptual overview of services and activities.

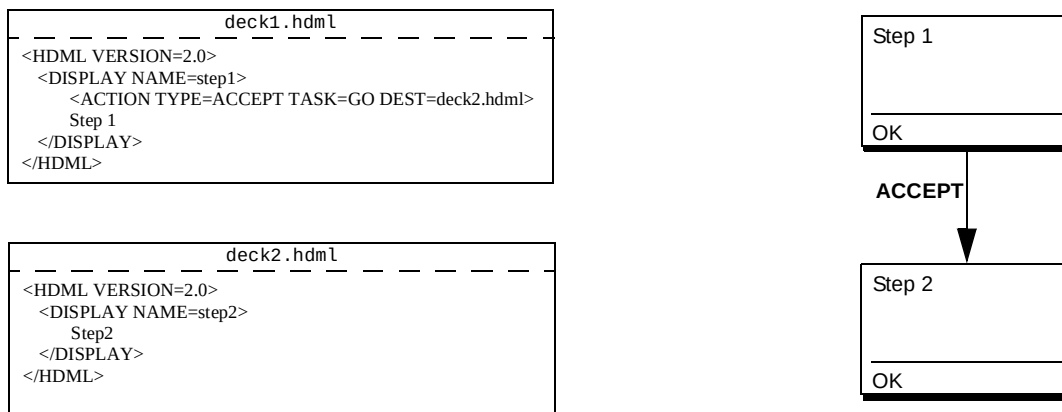
FIGURE 1-6. Services and activities



Each step in an activity typically corresponds to a single HDML card. For example, a “Get a stock quote” activity might include one card to enter the ticker symbol and another card to view the quote.

The HDML for an activity uses GO tasks (TASK=GO options) to navigate from one card to the next. You can implement an activity with a single deck (as shown in Figure 1-5) or multiple decks (as shown in Figure 1-7).

FIGURE 1-7. Activity spanning multiple decks

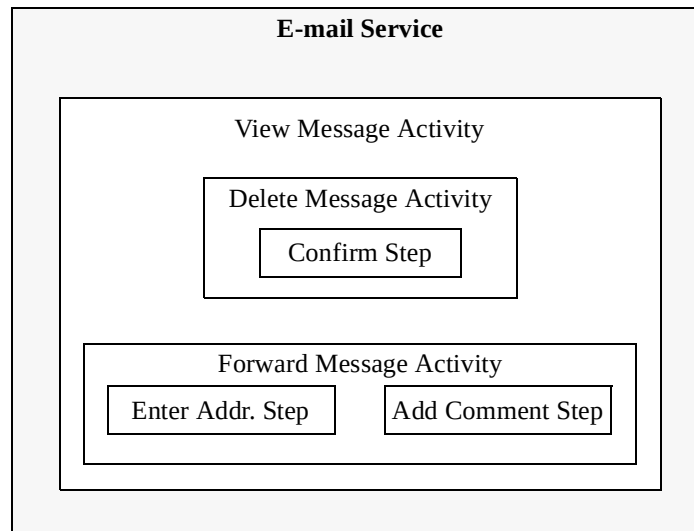


Nested activities

Just as a task can include subtasks, an activity can include nested “subactivities.” For example, suppose you want to implement a “View Message” activity of an email service. One subtask of the “View Message” task might be to delete the message; another might be to forward the message. To implement the “View Message” activity, you create HDML activities to match these tasks. You create a main activity, “View

Message,” and nested subactivities, “Delete Message” and “Forward Message.” Figure 1-8 depicts this implementation.

FIGURE 1-8. Nested activities



You can nest activities multiple levels deep. For example, the “View Message” activity described above could be nested within another activity, such as a “View Inbox” activity.

To spawn a nested activity in HDML, you specify a `GOSUB` task to go to the first card in the activity. For example, to implement the “View Inbox” activity described above, you would specify `GOSUB` tasks to navigate to the first cards of the “Read Message” and “Forward Message” activities.

Activity history

As the user navigates into nested activities, the phone preserves the *state* of each activity. The state includes attributes such as the activity’s variables and its card history stack.

Canceling an activity

HDML allows you to specify actions that `CANCEL` the current activity. Canceling an activity returns the user to the activity that originally invoked the current activity.

Variables

HDML *variables* are named pieces of information that the phone substitutes into HDML at runtime. They allow you to write HDML that dynamically changes display content and navigation based on user input. You can set variables explicitly, or you can allow the user to set them with choice or entry cards. You can substitute variable values into display text and certain HDML options such as the `DEST` (destination) option.

Variable naming

There are relatively few guidelines for variable naming:

- The following nonalphanumeric characters are not allowed:
\$, <, >, =, /, \, &, *, #
- Case is significant: `myvar` and `Myvar` are two different variables
- Variable names are stored in device memory, so they should be relatively short

Variable scope

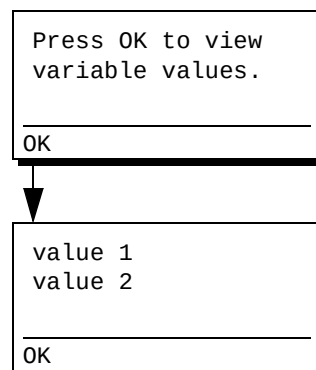
Variables are scoped to *activities*. You can reference a variable anywhere within the activity in which it is set; you can't reference it in other activities.

Setting variables

The `VARS` task option allows you to set a list of variables when you navigate from one card to another. For example, the following deck includes a display card which sets the value of two variables when the user presses `ACCEPT`. A second display card displays the variable values.

```
<HDML VERSION=2.0>
  <DISPLAY>
    <ACTION TYPE=ACCEPT TASK=GO DEST=#viewvalues
      VARS=var1=value+1&var2=value+2>
    Press OK to view variable values.
  </DISPLAY>
  <DISPLAY NAME=viewvalues>
    <WRAP>$var1
    <WRAP>$var2
  </DISPLAY>
</HDML>
```

FIGURE 1-9. Setting variables



Capturing user input to variables

You capture user input to a variable by specifying the variable's name in the `KEY` option of a choice or entry card. For example, the following HDML defines an entry card that stores the user's input to a variable named `username`.

```
<HDML VERSION=2.0 MARKABLE=false>
  <ENTRY NAME=nameentry KEY=username>
    <ACTION TYPE=ACCEPT TASK=GO DEST="#greeting">
      Your name:
    </ENTRY>
    . . .
```

Referencing variables

You can substitute variable values into the following:

- Formatted text
- Option values
- Choices
- Choice and entry card defaults

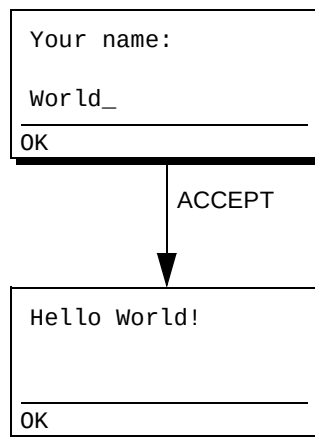
HDML 2.0 variable substitution is much like UNIX shell variable substitution. The following table describes the methods of substituting variable values:

Variable reference	Description
<code>\$myvar</code> or <code>\$(myvar)</code>	Substitutes the value of <code>myvar</code> . The phone attempts to “do the right thing”—depending on the context, it escapes nonalphanumeric characters according to URL conventions.
<code>\$(myvar:esc)</code>	Substitutes the value of <code>myvar</code> , escaping nonalphanumeric characters according to URL conventions.
<code>\$(myvar:noesc)</code>	Substitutes the value of <code>myvar</code> (<i>without</i> escaping characters). This method is useful for substituting variables that are already URL escaped.

For example, the following HDML allows the user to set a variable in an entry card; it then substitutes the variable's value in the display text of another card. Figure 1-10 displays the resulting interface.

```
<HDML VERSION=2.0 MARKABLE=false>
  <ENTRY NAME=nameentry KEY=username>
    <ACTION TYPE=ACCEPT TASK=GO DEST="#greeting">
      Your name:
    </ENTRY>
  <DISPLAY NAME=greeting>
    Hello $(username)!
  </DISPLAY>
</HDML>
```

FIGURE 1-10. Setting and substituting variable values



Using the `esc` and `noesc` options

In most cases, you do not need to specify the `esc` or `noesc` options. The phone substitutes the variable using the conversion appropriate for the context. Specifically, the phone uses URL escaping when you specify a variable in the following options:

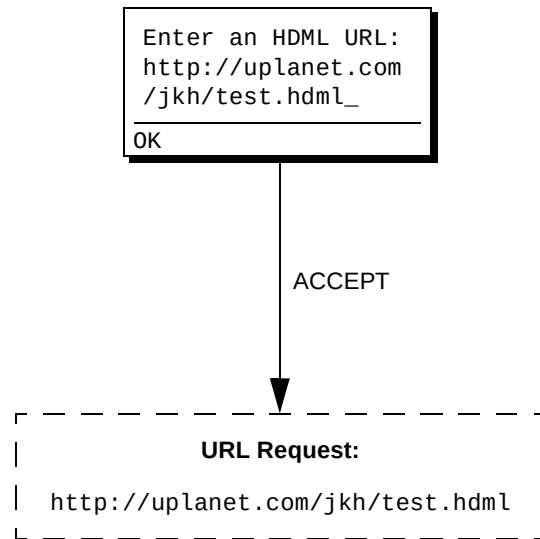
- `ACCESSDOMAIN`
- `ACCESSPATH`
- `BOOKMARK`
- `CANCEL`
- `DEST`
- `GO`
- `IMAGE`
- `NEXT`
- `SRC`

By default, the phone does not use URL escaping in any other context.

There are a few cases when you might need to specify the `esc` or `noesc` options. For example, If you use a variable to specify a full destination URL, you must use the `noesc` option. If you do not, the phone will include escape characters when it inserts the variable value, which will result in an invalid, doubly-escaped URL. For example, the following HDML displays a card that allows the user to enter the URL of an HDML service. When the user presses `ACCEPT`, the phone requests the URL that the user entered.

```
<HDML VERSION=2.0>
  <ENTRY KEY=desturl>
    <ACTION TYPE=ACCEPT TASK=GOSUB DEST="$(desturl:noesc)">
      Enter an HDML URL:
    </ENTRY>
  </HDML>
```

FIGURE 1-11. Setting and substituting variable values



If you did not use the `noesc` option to substitute the variable value, the phone would request the following, invalid URL:

```
Http%3a%2f%2fuplanet.com%2fjkh%2ftest.html
```

Passing variables between activities

HDML 2.0 provides several mechanisms to pass variable information between activities:

- An activity can use the `VARs` option (with the `GOSUB` task) to set variable values for a nested activity
- When an activity invokes a nested activity, it can use the `RECEIVE` option to specify variables that accept return values from the nested activity; the nested activity uses the `RETVALs` option to return values to these variables
- When a nested activity returns, it can specify the `CLEAR` option to unset all the variables of the activity that called it.

Formatted text

HDML allows you to display formatted text in display cards and display content areas of entry and choice cards. It provides a variety of statements to specify how text is wrapped and scrolled, and how it is placed on the phone display.

Reserved characters

The `<`, `>`, `"`, `$`, and `&` characters are reserved. To display one of these characters in formatted text, specify one of the following escape sequences.

Character	Escape sequence
<code><</code>	<code>&lt;</code>
<code>></code>	<code>&gt;</code>
<code>"</code>	<code>&quot;</code>
<code>&</code>	<code>&amp;</code>
<code>\$</code>	<code>&dol;</code>
Any ASCII character	<code>&#nn;</code> (where <code>nn</code> is the ASCII code)

IMPORTANT The semicolon (`;`) is part of the escape sequence for a special character. If you omit it, the UP.Phone will display an invalid deck error.

The UP.SDK provides a Perl utility function named `HDMLEscapeString()`, which converts the HDML reserved characters in a string to valid escape sequences. For more information, see the *UP.SDK Tools and APIs Reference* manual.

Line breaking, wrapping, and horizontal scrolling

You can use the following HDML statements to start new lines of formatted text.

Statement	Effect
<code>
</code>	Starts a new line. The line uses the default wrapping mode you last specified on the card.
<code><LINE></code>	Starts a new line. If the text is too large to fit the phone display, the phone automatically scrolls it horizontally so the user can view the entire line. This wrapping mode is known as “Times Square” mode. Until a <code><WRAP></code> statement is encountered, all lines that follow the current line are also scrolled.
<code><WRAP></code>	Starts a new line. If the text is too large to fit the phone display, the phone word wraps the text to the next line. Until a <code><LINE></code> statement is encountered, all lines that follow the current line are also word wrapped.

In `<LINE>` mode, the phone ignores white space such as multiple spaces, carriage returns, and newlines; these characters do not start new lines. In `<WRAP>` mode, the phone uses whitespace to conduct word wrapping.

`<WRAP>` is the default line wrapping mode for formatted text in display, entry, and choice cards. Formatted text in these cards uses the `<WRAP>` mode until a line starting

with a `<LINE>` statement is encountered. `<LINE>` is the default wrapping mode for the text in individual choice entry elements.

Vertical scrolling

If a card contains more lines of text than the phone can display, the phone displays as many lines as will fit on the display. Some phones may display visual cues to indicate to the user that there is additional text in the current card. The user can scroll down and see the remaining lines by pressing the down arrow key. After scrolling down, the user can scroll back to the first lines by pressing the up arrow key.

Tabs

To create aligned columns of text, insert `<TAB>` statements in the text. The phone automatically sets tab stops for you. It treats contiguous lines containing tabs as rows in a table, setting the number of columns to the number of tabs in the line with the most tabs. It sets the width of each column to accommodate the column's largest cell.

IMPORTANT Using tabs in centered lines has unpredictable results; text columns may run into each other.

Spaces

The HDML compiler converts multiple contiguous spaces, returns, and newlines into a single space. To display multiple contiguous spaces, insert nonbreaking spaces with the ` ` element. Each ` ` element has the width of a regular space. Note that if you want to allow wrapping in the middle of a series of contiguous nonbreaking spaces, you must insert a regular space where you want to allow wrapping. If you don't want the phone to wrap a line on a particular space, replace the space with a ` ` element.

Alignment

By default, formatted text is left aligned. To change the alignment, you can use the following statements at the beginning of a line.

Code	Effect
<code><RIGHT></code>	Text is right aligned.
<code><CENTER></code>	Text is centered.

A `<RIGHT>` or `<CENTER>` statement applies to the entire line.

Example

The following HDML deck generates the display shown in Figure 1-12:

```
<HDM1 VERSION=2.0>
  <DISPLAY>
    <CENTER>Average Rainfall
    <LINE><TAB>March<TAB>April
    <LINE>TVL<TAB>10"<TAB>12";
    <LINE>ORD<TAB>8"<TAB>6";
  </DISPLAY>
</HDM1>
```

FIGURE 1-12. Display card with formatted text containing tabs

Average Rainfall	
March	April
TVL 10"	12"
OK	

The phone display in this example uses a vertical bar (|) as visual cue to indicate there is additional text to which the user can scroll.

HDML Statement Reference

2

This chapter provides reference information for HDML statements and options. The statements and options are listed in alphabetical order.

Actions (<ACTION>)

An action associates a task with an UP.Phone function key (ACCEPT, PREV, or HELP, SOFT1-SOFT2). When the user presses the function key, the UP.Phone executes the task.

The following tasks are currently supported:

- Requesting the current URL with different arguments
- Requesting another URL
- Requesting another URL, starting a nested activity
- Returning to the calling activity
- Cancelling the current activity, returning to the calling activity
- Displaying the previous card in the history
- Dialing a specified phone number

Action precedence for function keys

You can specify actions and tasks at several levels in HDML. For example, both a card and the deck that contains it can include actions that specify tasks for ACCEPT. The UP.Phone uses the rules of precedence described in the sections below to determine which task to execute.

ACCEPT

When the user presses ACCEPT, the UP.Phone uses the following rules to determine what action to take:

- 1 If the current card is a choice card and the choice entry the user selected specifies a task, it executes the task and skips the following rules.
- 2 If the current *card* defines an action for ACCEPT, it executes the task specified by the action and skips the following rules.
- 3 If the current *deck* defines an action for ACCEPT, it executes the task specified by the action and skips the following rule.
- 4 Displays the previous card.

PREV

When the user presses PREV, the UP.Phone uses the following rules to determine what action to take:

- 1 If the current *card* defines an action for PREV, it executes the task specified by the action and skips the following rules.
- 2 If the current *deck* defines an action for PREV, it executes the task specified by the action and skips the following rule.
- 3 Displays the previous card. If the current card is the first card in the current activity, the UP.Phone cancels the current activity and returns to the card that originally invoked it.

SOFT1-SOFT2

The UP.Phone handles the softkeys, SOFT1-SOFT2, in a similar fashion. The rest of this manual uses the term SOFTx to describe behavior that applies to SOFT1-SOFT2.

When the user presses SOFTx, the UP.Phone uses the following rules to determine what action to take:

- 1 If the current *card* defines an action for SOFTx, it executes the task specified by the action and skips the following rules.
- 2 If the current *deck* defines an action for SOFTx, it executes the task specified by the action and skips the following rules.
- 3 Nothing.

If you have specified an action for a key at the deck level but you want the key to be inactive in an individual card, specify an action with no task for the card.

HELP

When the user presses HELP, the UP.Phone uses the following rules to determine what action to take:

- 1 If the current *card* defines an action for HELP, it executes the task specified by the action and skips the following steps.
- 2 If the current *deck* defines an action for HELP, it executes the task specified by the action.

- 3 Displays a built-in card containing a message that no help is available for the current location.

Syntax

Statement	Description
<ACTION	Beginning of the ACTION statement.
<i>LABEL=keylabel</i>	The label for the key. <i>keylabel</i> should be five characters or shorter. If you are associating a task with the ACCEPT key, the label is optional. If you are associating a task with the PREV key, any label you specify is ignored. Otherwise, the label is required. The default label for the ACCEPT key is OK.
<i>TYPE=key</i>	The key with which to associate a task. Possible values are: ACCEPT HELP PREV SOFT1 SOFT2 For a description of the default actions and action precedence for each of these keys, see “Action precedence for function keys” on page 27.
<i>TASK=tasktype</i>	The task to execute. Possible values are: GO: requests the URL specified by the DEST option. GOSUB: requests the URL specified by the DEST option, spawning a nested activity. RETURN: returns from a nested activity to the previous activity. CANCEL: cancels the current nested activity, returning to the previous activity. PREV: displays the previous card in the activity history. If the current card is the first card, PREV has the same effect as CANCEL. CALL: places a phone call to the number specified by the number option. NOOP: does nothing. See the table below for more detailed descriptions of these tasks and the options you can use with them.

Statement	Description
<i>DEST=desturl</i>	The URL to request in <code>GO</code> and <code>GOSUB</code> tasks. If the current activity is nested and the task is a <code>RETURN</code> or <code>CANCEL</code> task, <code>DEST</code> specifies a URL to request upon returning to the calling activity. However, if the calling activity does not designate the current activity as “friendly,” the UP.Phone ignores the <code>DEST</code> option. The <code>DEST</code> option overrides the <code>NEXT</code> or <code>CANCEL</code> options of the calling activity.
<i>VARS=varpairs</i>	A list of variables to set for the current activity (if the task is <code>GO</code>) or nested activity (if the task is <code>GOSUB</code>). The variable list must be in query-string format, for example: <code>VAR1=value1&VAR2=value2</code> Variable values must be escaped according to URL escaping conventions. The phone unescapes the <code>VARS</code> option before setting the value of the variables.
<i>RECEIVE=varlist</i>	A semicolon-delimited list of variables to which the phone stores the return values from a <code>GOSUB</code> task. The phone stores return values according to ordinal position. For example, if you specify the following option: <code>RECEIVE=var1;var2</code> the phone stores the first return value to <code>var1</code> and the second return values to <code>var2</code>. If you want to skip a return value, you must include a semicolon as a placeholder. For example, to ignore the first return value and store the second return value to <code>var2</code>, you specify the following option: <code>RECEIVE=;var2</code>
<i>RETVALS=vallist</i>	A semicolon-delimited list of values that an activity invoked with <code>GOSUB</code> returns to the invoking activity. The <code>RETVALS</code> option is allowed only with the <code>RETURN</code> task. The values must be escaped according to URL escaping conventions.
<i>NEXT=nextdest</i>	The URL to request after a nested activity returns. If the <code>FRIEND</code> option in the <code>GOSUB</code> task is set to <code>TRUE</code>, the <code>NEXT</code> option can be overridden by the <code>DEST</code> option in the nested activity’s <code>RETURN</code> task.
<i>CANCEL=canceldest</i>	The URL to request after a nested activity invoked by a <code>GOSUB</code> task cancels. If the <code>FRIEND</code> option in the <code>GOSUB</code> task is set to <code>TRUE</code>, the <code>CANCEL</code> option can be overridden by the <code>DEST</code> option in the nested activity’s <code>CANCEL</code> task.

Statement	Description
<i>FRIEND=boolean</i>	A boolean value specifying whether the nested activity specified in a GOSUB task is “friendly.” A friendly nested activity can use the DEST and CLEAR options in RETURN and CANCEL tasks and override the NEXT and CANCEL options of the calling task. To indicate the nested activity is friendly, specify TRUE . The default is FALSE .
<i>SENDREFERER=boolean</i>	A boolean value specifying whether the UP.Browser should provide the URL of the current deck when requesting the URL specified by the DEST or NEXT options. If you set it to TRUE , the UP.Browser specifies the deck’s URL in the “Referer” header of the request. The UP.Browser attempts to use the shortest possible relative URL in the “Referer” header if possible. The default value is FALSE .
<i>CLEAR=boolean</i>	A boolean value specifying whether a RETURN or CANCEL task from a nested activity unsets all the calling activity’s variables. To unset the calling activity’s variables, specify TRUE . The default is FALSE . The phone ignores the CLEAR option unless the calling activity specifies that the current activity is “friendly.”
<i>NUMBER=number</i>	For a CALL task, the phone number to call.
<i>IMAGE=url</i>	The URL of an image to display above the softkey.
>	End of the ACTION statement.

Task types

The following table lists the task types you can specify for the **TASK** option. It also lists the other options you can use with each task type. Required options appear in bold.

Task	Description	Options
GO	Requests the URL specified by the DEST option. If you use the GO task, you can only specify a relative URL for the DEST option.	DEST , VARS , SENDREFERER
GOSUB	Pushes a new activity onto the activity stack and requests the URL specified by the DEST option. When the nested activity returns, the phone puts the nested activity’s return values into the variables specified by the RECEIVE option. If the nested activity cancels, the phone requests the URL specified by the CANCEL option.	DEST , VARS , SENDREFERER , FRIEND , RECEIVE , NEXT , CANCEL

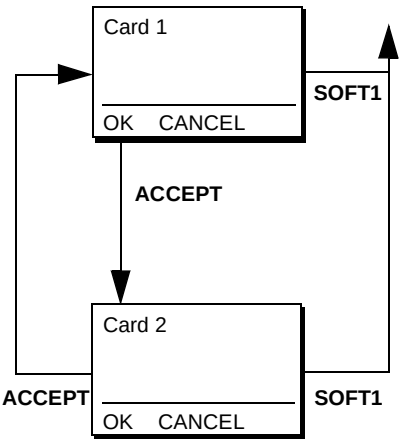
Task	Description	Options
PREV	Displays the previous card in the activity history. If the current card is the first card in the current activity, PREV has the same effect as CANCEL.	
RETURN	Returns from a nested activity to the previous activity with the return values specified by the RETVALS option.	RETVALS, DEST, CLEAR
CANCEL	Cancels the current activity, requesting the URL specified by the previous activity's CANCEL option. If no CANCEL option is specified, the phone requests the current card in the previous activity.	DEST, CLEAR
CALL	Switches the phone to voice mode and dials the number specified by the NUMBER option.	NUMBER
NOOP	Do nothing. This is useful for disabling the default behavior of the specified action.	

Examples

The following code defines a deck with two display cards. The deck defines an action which assigns the CANCEL task and a label to the SOFT1 key; this action applies to both cards in the deck. The first card defines an action which specifies that the phone will request the second card when the user presses ACCEPT. The second card does not define an action for the ACCEPT key, so the phone implements the default (go to the previous card in the history). Neither card defines an action for the PREV key, so the phone uses the default behavior (go to the previous card in the history). Figure 2-1 summarizes the user interface.

```
<HDML VERSION=2.0>
  <ACTION TYPE=SOFT1 TASK=CANCEL LABEL=CANCEL>
  <DISPLAY NAME=card1>
    <ACTION TYPE=ACCEPT TASK=GO DEST=#card2>
    Card 1
  </DISPLAY>
  <DISPLAY NAME=card2>
    Card 2
  </DISPLAY>
</HDML>
```


FIGURE 2-1. HDML deck with two display cards



Choice cards (<CHOICE>)

Choice cards display some text followed by a list of items from which the user can choose a single item, called a *choice entry*. To define a choice card, you use a `<CHOICE>` statement. The statement must include one or more `<CE>` statements defining the items on the list.

Syntax

<code><CHOICE</code>	Beginning of the choice card header.
<code>NAME=card_name</code>	A name for the card. Other cards can specify this name as a destination.
<code>MARKABLE=boolean</code>	Flag specifying whether the card can be marked. To allow the card to be marked, specify <code>TRUE</code>. The default value is determined by the <code>MARKABLE</code> option for the deck (see “Decks (<HDML>)” on page 37). Always set the <code>MARKABLE</code> option to <code>FALSE</code> for cards that execute or confirm transactions that the user might not want to repeat. If you set the <code>MARKABLE</code> option to <code>TRUE</code>, it implicitly sets the <code>PUBLIC</code> option to <code>TRUE</code>, allowing other URLs to link directly to the card.
<code>TITLE=cardtitle</code>	The default bookmark name that appears when the user marks the card.

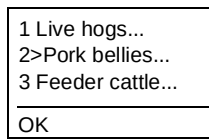
BOOKMARK=markURL	The URL the phone adds to the bookmark list if the user marks the card. If you do not specify the BOOKMARK option, the phone adds the URL of the current card to the bookmark list.
KEY=varname	The name of the variable that gets the value (if any) of the chosen item.
IKEY=varname	The name of a variable containing an index number. The phone uses the index number to set the default item. The number 1 specifies the first item, 2 specifies the second item, and so on. For example, if <i>var_name</i> is <i>myvar</i> and the value of <i>myvar</i> is 2, the second item is the default item. If you don't specify the IKEY option or the IDEFAULT option, the first item is the default. When the user chooses an item, the phone sets the variable specified by IKEY to the number of the chosen item.
METHOD=choice_method	The choice method. To make the list numbered, specify NUMBER. If the list is numbered, the user can choose an item by pressing the item's number or by scrolling to it and pressing a key. To make the list unnumbered, specify ALPHA. If the list is unnumbered, the user can choose an item only by scrolling to it and pressing ACCEPT. If you do not specify the METHOD option at all, the choice method defaults to NUMBER. Note that the phone will number only the first nine items in the list.
DEFAULT=defaultval	The default value of the variable specified by the KEY option. If the variable specified by the KEY option does not have a value when the card is first displayed, the phone assigns it the value specified by the DEFAULT option. If the variable specified by the KEY option already has a value, the DEFAULT option is ignored.
IDEFAULT=defaultnum	The index of the default entry, if the IKEY option is not specified or it specifies a variable with no value. If the IKEY option is specified, IDEFAULT is ignored.
>	End of the choice card header.

<i>actions</i>	The actions to execute when the user presses a function key. If the choice item the user chooses specifies a task, the phone executes the task instead of these actions. For more information on specifying actions, see “Actions (<ACTION>)” on page 27.
<i>display_content</i>	A formatted text title to display above the list of items. It should be as brief as possible. Avoid using the <LINE> tag for this text; if you use it, text that exceeds the display width will be truncated. For more information on specifying formatted text, see “Formatted text” on page 23.
<CE VALUE=val task>text	An item the user can choose. <i>val</i> is an optional value the UP.Phone stores to the variable specified by the KEY option when the user chooses the item. <i>task</i> is an optional task to execute when the user chooses the item. It can include task options. If you specify a task for the item the user chooses, the phone executes the task instead of any actions you have specified at the card or deck level. For a description of the task types, see “Task types” on page 31. <i>text</i> is a formatted text line that the UP.Phone displays on the item line. If you did not specify ALPHA for the choice method, the item’s number is automatically prepended to the text. By default, the UP.Phone always displays the text in horizontal scrolling (<LINE>) mode. To make it wrap, insert a <WRAP> statement before the text.
. . .	Additional items (<CE> statements) as needed.
</CHOICE>	The choice card footer.

The following HDML deck generates the display shown in Figure 2-2:

```
<HDML VERSION=2.0>
  <CHOICE KEY=ch IKEY=num IDEFAULT=2>
    <ACTION TYPE=ACCEPT TASK=GO
      DEST="?choiceval=$ch&choicenum=$num">
      <CE VALUE=Hogs>Live hogs...
      <CE VALUE=Bellies>Pork bellies...
      <CE VALUE=Cattle>Feeder cattle...
    </CHOICE>
  </HDML>
```

FIGURE 2-2. Choice card



If the user presses 3 and then presses ACCEPT, the current URL is called with the following arguments:

?choiceval=Cattle&choicenum=3

The phone sets the variables specified by the IKEY and IDEFAULT options according to which choice item is selected when the user leaves the card. The phone always sets these variables, regardless of the softkey the user presses to leave the card.

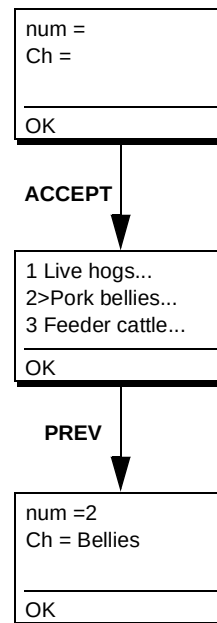
IMPORTANT The phone sets the variables specified by the IKEY and IDEFAULT options even if the user presses a key other than ACCEPT to leave a choice card. For example, the phone sets the variables even if the user presses PREV.

Suppose we modify the previous example deck so that the first card displays the variables specified by IKEY and IDEFAULT.

```
<HDML VERSION=2.0>
  <DISPLAY>
    <ACTION TYPE=ACCEPT TASK=GO DEST=#chcard>
      num = $num <BR>
      ch = $ch
    </DISPLAY>
  <CHOICE KEY=ch IKEY=num IDEFAULT=2 NAME=chcard>
    <ACTION TYPE=ACCEPT TASK=GO
      DEST="?choiceval=$ch&choicenum=$num">
      <CE VALUE=Hogs>Live hogs...
      <CE VALUE=Bellies>Pork bellies...
      <CE VALUE=Cattle>Feeder cattle...
    </CHOICE>
  </HDML>
```

If the user goes to the choice card and then immediately presses PREV to return to the first card, the first card displays the variable values as shown in Figure 2-3.

FIGURE 2-3. Pressing keys other than ACCEPT in a choice card



Comments

HDML allows you to add comments to HDML code. The UP.Phone ignores comments.

Syntax

<code><-- comment_text --></code>	<i>comment_text</i> is the comment text
<code><!-- comment_text --></code>	<i>comment_text</i> is the comment text

Examples

The following are comments:

```

. . .
<-- This is a comment -->
<!-- This is a comment -->
. . .

```

Decks (<HDML>)

A deck is used to transport and cache sets of HDML cards. It is the smallest unit of HDML a service can transmit to the UP.Link, and the only HDML unit for which you

can specify a Time To Live (TTL)—the period during which the phone caches the deck.

You can also use a deck to group cards for which you want to set the same bookmarking and access settings.

To define a deck, you use an <HDML> statement. The statement must include one or more statements defining display, choice, or entry cards. It can also specify options and actions that apply to the entire deck.

Syntax

<HDML	Beginning of the deck header.
VERSION=version_num	The version of the HDML language used for the deck. The syntax is <i>maj.min.lex</i>, where <i>maj</i> is the major version, <i>min</i> is the minor version, and <i>lex</i> is the lexical version. The lexical version number is optional. Valid version numbers are 0.1 and 2.0. The current version, documented in this manual, is 2.0. You must specify a version number. If you do not, the HDML compiler generates an error message.
TTL=cache_time	The number of seconds the UP.Phone should cache the deck after it receives it. If you do not specify a value, the UP.Phone caches the deck for 30 days. If <i>cache_time</i> is 0, the UP.Phone does not cache the deck at all.
MARKABLE=boolean	Flag specifying whether the cards in the deck can be marked. To allow the cards to be marked, specify TRUE. The default value is FALSE. If you set the MARKABLE option to TRUE, it implicitly sets the PUBLIC option to TRUE, allowing other URLs to link directly to cards within the deck. If the statement for a card within the deck specifies the MARKABLE option, it overrides the deck's MARKABLE option.
PUBLIC=boolean	Specifies whether deck access control is enabled. If you specify TRUE, any deck can link to cards within the deck. If you specify FALSE, only decks that have domains and paths specified by the ACCESSDOMAIN and ACCESSPATH options can link to cards within the deck. The default is FALSE.

<i>ACCESSDOMAIN=domain</i>	Specifies the domain of URLs that are allowed to request cards in the deck if PUBLIC is set to FALSE. The default value is the current deck's domain. For a detailed explanation and examples of how the ACCESSPATH and ACCESSDOMAIN options work together, see the <i>UP.SDK Developer's Guide</i>.
<i>ACCESSPATH=path</i>	Specifies the base path of URLs that are allowed to request cards within the deck when PUBLIC is set to FALSE. The default value is / (the root of the current deck's path); this means that any deck with the domain specified by ACCESSDOMAIN can request the deck. For a detailed explanation and examples of how the ACCESSPATH and ACCESSDOMAIN options work together, see the <i>UP.SDK Developer's Guide</i>.
>	End of the deck header.
<i>actions</i>	Actions that apply to the entire deck.
<i>cards</i>	One or more cards. A deck must contain at least one card. For more information on defining cards see "Choice cards (<CHOICE>)" on page 33 and "Display cards (<DISPLAY>)" on page 41.
</HDML>	The deck footer.

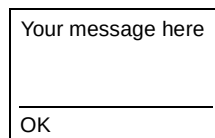
IMPORTANT The UP.Link cannot transmit *compiled* HDML decks or digests larger than 1492 bytes to UP.Phones. If you send a deck larger than 1492 bytes to the UP.Link, the UP.Link generates an error. Because compiling a deck can increase or decrease its size slightly, uncompiled decks slightly smaller than 1492 bytes may also fail. To be safe, you should attempt to keep all uncompiled decks smaller than 1200 bytes.

Examples

The following HDML statements generate the display shown in Figure 2-4:

```
<HDML VERSION=2.0>
  <DISPLAY>
    <WRAP>Your message here.
  </DISPLAY>
</HDML>
```

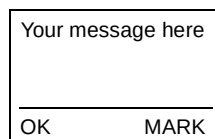
FIGURE 2-4. Deck with a single display card



Suppose you want to make sure the UP.Phone does not cache the card. You also want to allow the user to mark the deck. To do this, you change the HDML statements to the following:

```
<HDML VERSION=2.0 MARKABLE=TRUE TTL=0>
  <DISPLAY>
    <WRAP>Your message here.
  </DISPLAY>
</HDML>
```

When you specify the `MARKABLE=TRUE` option, the mark (MARK) label appears as shown in Figure 2-5. Note that some phones do not display the mark label.

FIGURE 2-5. Display card with the `MARKABLE=TRUE` option

Destinations (DEST=)

A destination identifies the deck or card to display when you specify an action. You can identify a deck with a full, partial, or relative path. You identify a card with its name.

Syntax

HDML 2.0 uses the same destination syntax as conventional URLs (as specified in RFC 1808 and RFC 1738).

The 2.0 UP.Browser understands both HTTP and HTTPS addressing schemes (and the *httpurl* syntax specified in RFC 1738). If no scheme is specified, the UP.Browser assumes the HTTP addressing scheme.

The following symbols still have special meanings when you use them in a path.

Symbol	Meaning
?	The URL of the current deck, but with no arguments
?ARG1=value ARG2=value...	The URL of the current deck with the specified arguments

The UP.Phone converts all relative URLs to absolute URLs before transmitting them to the UP.Link.

Destination examples

For a full description of URL syntax, see the RFC 1808, RFC 1738. The following are some simple examples.

Destination	Meaning
<code>http://uplanet.com/adir/deck.html</code>	An absolute path identifying the deck <code>deck.html</code> .
<code>http://uplanet.com/adir/deck.html?ARG1=2</code>	An absolute path identifying the deck <code>deck.html</code> with the argument <code>arg1</code> set to <code>2</code> .
<code>http://uplanet.com/adir/deck.html#acard</code>	An absolute path identifying the card named <code>acard</code> in the <code>deck.html</code> deck.
<code>#acard</code>	A relative path identifying <code>acard</code> . This destination is meaningful only if you specify it in the deck containing <code>acard</code> .
<code>otherdeck.html</code>	A relative path identifying a deck in the same directory.
<code>subdir/yad.html</code>	A relative path identifying a deck in the <code>subdir</code> subdirectory of the same directory.
<code>../deck.html</code>	A relative path identifying a deck in a directory one level up from the directory containing the current deck. Additional <code>../</code> strings can be prepended to specify directories further up the directory tree.

Display cards (<DISPLAY>)

Display cards display formatted text. To define a display card, you use a `<DISPLAY>` statement.

Syntax

<DISPLAY	Beginning of the display header.
<i>NAME=card_name</i>	A name for the card. Other cards can specify this name as a destination.
<i>TITLE=cardtitle</i>	The default bookmark name that appears when the user marks the card.

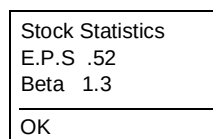
<i>MARKABLE</i> = <i>boolean</i>	Flag specifying whether the card can be marked. To allow the card to be marked, specify TRUE . The default value is determined by the MARKABLE option for the deck (see “Decks (<HDML>)” on page 37). Always set the MARKABLE option to FALSE for cards that execute or confirm transactions that the user might not want to repeat. If you set the MARKABLE option to TRUE , it implicitly sets the PUBLIC option to TRUE , allowing other URLs to link directly to the card.
<i>BOOKMARK</i> = <i>markURL</i>	The URL the phone adds to the bookmark list if the user marks the card. If you do not specify the BOOKMARK option, the phone adds the URL of the current card to the bookmark list.
>	End of the display header.
<i>actions</i>	The actions to execute when the user presses a function key. For more information on specifying actions, see “Actions (<ACTION>)” on page 27.
<i>text</i>	The formatted text the card displays. For more information on specifying formatted text, see “Formatted text” on page 23.
</DISPLAY>	The display footer.

Examples

The following HDML deck displays the card shown in Figure 2-6:

```
<HDML VERSION=2.0>
  <DISPLAY>
    <CENTER>Stock Statistics
    <LINE>E.P.S.<TAB>.52
    <LINE>Beta<TAB>1.3
  </DISPLAY>
</HDML>
```

FIGURE 2-6. Display card



Entry cards (<ENTRY>)

An entry card displays content followed by an entry line in which the user enters data. To define an entry card, you use an <ENTRY> statement. The statement specifies the format for data entered by the user and a variable to which the value in the field is stored. It can also specify a default value.

Syntax

<ENTRY	Beginning of the entry header.
NAME= <i>card_name</i>	A name for the card. Other cards can specify this name as a destination.
MARKABLE= <i>boolean</i>	Flag specifying whether the card can be marked. To allow the card to be marked, specify TRUE. The default value is determined by the MARKABLE option for the deck (see “Decks (<HDML>)” on page 37). Always set the MARKABLE option to FALSE for cards that execute or confirm transactions that the user might not want to repeat. If you set the MARKABLE option to TRUE, it implicitly sets the PUBLIC option to TRUE, allowing other URLs to link directly to the card.
TITLE= <i>cardtitle</i>	The default bookmark name that appears when the user marks the card.
BOOKMARK= <i>markURL</i>	The URL the phone adds to the bookmark list if the user marks the card. If you do not specify the BOOKMARK option, the phone adds the URL of the current card to the bookmark list.

FORMAT=*format_specifier*

A format specifier for the data the user enters. It uses a combination of the following:

A: any uppercase alphabetic character or punctuation (no numbers)

a: any lowercase alphabetic character punctuation (no numbers)

N: any numeric character (no symbols)

X: any numeric, symbolic, or uppercase alphabetic character or punctuation

x: any numeric, symbolic, or lowercase alphabetic character or punctuation

M: any alphabetic character (of any case), and any numeric or symbolic character. If it is preceded by a number or * specifier, the first character the user enters is uppercase by default (the user can change it to lowercase)

m: any alphabetic character (of any case), and any numeric or symbolic character. If it is preceded by a number or * specifier, the first character the user enters is lowercase by default (the user can change it to uppercase)

To allow a number of instances of a particular character, precede the character with a single digit number. To allow unlimited instances, precede it with an asterisk (*). For example, the specifier 3X allows the user to enter three numeric uppercase or alphabetic characters; *X allows the user to enter any number of numeric uppercase or alphabetic characters. Note that you can use this type of specifier (e.g. 3X or *X) only once in a formatting string and only at the end of the string.

You can also use the format specifier to automatically insert characters into the entry field. Precede each character you want to automatically insert with a backslash.

The default format is *M (any number of mixed case alphabetic and numeric characters).

The **FORMAT** option is discussed more in the text below.

DEFAULT = <i>default_value</i>	A string that appears in the entry field when the UP.Phone first displays the card. The user can edit the string. The default value can be formatted text or a variable containing text. If the variable specified by the KEY option has a value, it overrides the value specified by the DEFAULT option.
KEY = <i>var</i>	The name of the variable to which the UP.Phone stores the data entered by the user. If the specified variable already has a value, the value appears as the default in the entry field.
NOECHO = <i>boolean</i>	If NOECHO is set to TRUE, the phone hides text the user enters. After the user enters a character, the phone displays it as an asterisk (*) The default is FALSE.
EMPTYOK = <i>boolean</i>	If you set EMPTYOK to TRUE, the phone accepts empty input, even if you have specified a format with the FORMAT option. The default is FALSE.
>	End of the entry header.
<i>actions</i>	The actions to execute when the user presses a function key. For more information on specifying actions, see “Actions (<ACTION>)” on page 27.
<i>display_content</i>	The formatted text message to display above the entry line. For more information on specifying formatted text, see “Formatted text” on page 23.
</ENTRY>	The entry footer.

To instruct the UP.Phone to automatically add characters to the entry field, include them in the format string. Precede each character with a backslash. As the user reaches the position of each character, the UP.Phone adds it automatically. For example, if you specify the following string:

```
\(NNN\) \ NNN\ -NNNN
```

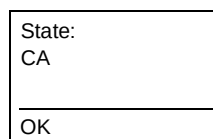
the UP.Phone adds the left parenthesis to the entry field immediately. After the user types three characters, the UP.Phone automatically adds the right parenthesis and a space. Users can not edit the automatically added characters.

Examples

The following HDML deck generates the display shown in Figure 2-7:

```
<HDML VERSION=2.0>
  <ENTRY KEY=state FORMAT=AA DEFAULT=CA>
    <ACTION TYPE=ACCEPT TASK=GOSUB DEST=?state=$state>
      State:
    </ENTRY>
</HDML>
```

FIGURE 2-7. Entry card display



If the user enters CA as shown in Figure 2-7 and presses ACCEPT, the current URL is requested with the following arguments:

?state=CA

Images ()

An statement specifies an image that appears formatted text. Not all phone models are capable of displaying images.

Statement	Description
<IMG	Beginning of the image header.
NAME = <i>img_name</i>	A name for the image. Some devices support built-in images.
ALT = <i>alttext</i>	Text to display if the image can't be displayed (if the phone does not support images or the image content can't be found).
SRC = <i>dest</i>	The image's URL.
>	
	The image footer.

No-display cards (<NODISPLAY>)

No-display cards do not appear on the phone. Instead, they immediately execute the ACCEPT or PREV action. They are useful for initializing variables.

Statement	Description
<NODISPLAY	Beginning of the No-Display header.
<i>NAME=card_name</i>	A name for the card. Tasks can specify this name as a destination.
>	
<i>actions</i>	In general, the phone automatically executes the ACCEPT actions when the user navigates forward to the card and the PREV actions when the user navigates back to the card. For the exact behavior with specific task types, see the table below. The phone executes only the ACCEPT and PREV actions defined for the no-display card; it ignores all other actions.
</NODISPLAY>	The no-display header.

The following table lists the task types that can invoke a no-display card and the actions the phone executes in the no-display card.

Invoking task type	No-display action executed by phone
GO	ACCEPT
GOSUB	ACCEPT
RETURN	PREV
PREV	PREV
CANCEL	PREV
RETURN or CANCEL with DEST specifying the No-Display card	ACCEPT
RETURN with the NEXT option of the calling card specifying the no-display card	ACCEPT
RETURN with the CANCEL option of the calling card specifying the no-display card	ACCEPT

HDML Quick Reference

This chapter lists HDML statements and keywords by topic and provides a brief synopsis of each statement. For a detailed description of a specific statement, see the page number next to the topic.

Defining cards and decks

To	Use
Define a deck (page 37)	<pre> <HDML VERSION=<i>hdml_version</i> TTL=<i>cache_time</i> MARKABLE=<i>boolean</i> PUBLIC=<i>boolean</i> ACCESSDOMAIN=<i>url</i> ACCESSPATH=<i>path</i>> actions cards </HDML></pre>
Define a Choice card (page 33)	<pre> <CHOICE NAME=<i>card_name</i> MARKABLE=<i>boolean</i> TITLE=<i>card_title</i> BOOKMARK=<i>mark_URL</i> KEY=<i>varname</i> IKEY=<i>varname</i> METHOD=<i>choice_method</i> DEFAULT=<i>default_val</i> IDEFAULT=<i>defaultnum</i>> actions prompt_text <CE VALUE=<i>val1</i> task> <i>text1</i> <CE VALUE=<i>val2</i> task> <i>text2</i> . . . </CHOICE></pre>

To	Use
Define a Display card (page 41)	<DISPLAY <i>NAME=card_name</i> <i>MARKABLE=boolean</i> <i>TITLE=card_title</i> <i>BOOKMARK=mark_URL</i> > <i>actions</i> <i>display_text</i> </DISPLAY>
Define an Entry card (page 43)	<ENTRY <i>NAME=card_name</i> <i>MARKABLE=boolean</i> <i>TITLE=card_title</i> <i>BOOKMARK=mark_URL</i> <i>FORMAT=format_specifier</i> <i>DEFAULT=default_value</i> <i>KEY=var</i> <i>NOECHO=boolean</i> <i>EMPTYOK=boolean</i> > <i>actions</i> <i>prompt_text</i> </ENTRY>
Define a No-Display card (page 47)	<NODISPLAY <i>NAME=card_name</i> > <i>actions</i> </NODISPLAY>

Displaying formatted text

To	Use
Center text	<CENTER> <i>text line</i>
Right align text	<RIGHT> <i>text line</i>
Nonbreaking space	&nbsp;
Tab	<TAB>
Left angle bracket (<)	&lt;
Right angle bracket (>)	&gt;
Quotation marks(")	&quot;
Ampersand (&)	&amp;
Dollar sign (\$)	&dol;
Start a new line using wrapping mode	<WRAP>

To	Use
Start a new line using “Times Square” horizontal scrolling mode	<LINE>
Start a new line using the current mode	

Specifying actions and tasks

To	Use
Specify an action (page 27)	<ACTION <i>TYPE=key_name</i> <i>LABEL=key_label</i> <i>task</i> <i>IMAGE=url</i> >
Define a Choice card, with tasks for individual choices (page 33)	<CHOICE <i>card_options</i> <i>actions</i> <i>prompt_text</i> <CE VALUE=val1 task1> <i>text1</i> <CE VALUE=val2 task2> <i>text2</i> <i>. . .</i> </CHOICE>
Specify a GO task (page 27)	TASK=GO DEST=dest_URL VARs=varpairs SENDREFERER=boolean
Specify a GOSUB task (page 27)	TASK=GOSUB DEST=dest_URL VARs=varpairs RECEIVE=var_list NEXT=nextdest CANCEL=cancel_dest FRIEND=boolean SENDREFERER=boolean
Specify a PREV task (page 27)	TASK=PREV
Specify a RETURN task (page 27)	TASK=RETURN DEST=dest_URL RETVALs=value_list CLEAR=boolean

To	Use
Specify a CANCEL task (page 27)	TASK =CANCEL DEST = <i>dest_URL</i> CLEAR = <i>boolean</i>
Specify a CALL task (page 27)	TASK =CALL NUMBER = <i>number</i>
Specify a NOOP task (page 27)	TASK =NOOP

Specifying destinations

To	Use
Specify a deck relative to the URL of the current deck	/deck_path or deck_path
Specify another card in the current deck	# <i>card_name</i>
Specify the URL of the current deck with no options	?
Specify the URL of the current deck with the specified arguments	? <i>arg1=value arg2=value...</i>
Specify a device-provided service	device : <i>device_service_path</i>

Defining images

To	Use
Specify an image (page 46)	<IMG ALT = <i>alttext</i> NAME = <i>image_name</i> SRC = <i>dest</i>

Index

Symbols

\$ character, escape sequence for 24

& escape sequence 24

&dol; escape sequence 24

> escape sequence 24

< escape sequence 24

" escape sequence 24

' character, escape sequence for 24

< character, escape sequence for 24

 statement (image) 46

<NODISPLAY> statement (no-display) 47

> character, escape sequence for 24

" character, escape sequence for 24

A

ACCESSDOMAIN option (<HDML> statement) 39

ACCESSPATH option (<HDML> statement) 39

<ACTION> statement 27

actions 27

precedence and scope 15, 27

activities 17

history 19

nested 18

alignment, of displayed text 25

ALT option (statement) 46

arguments

in destinations 19

B

BOOKMARK option (<CHOICE> statement) 34

BOOKMARK option (<DISPLAY> statement) 42

BOOKMARK option (<ENTRY> statement) 43

 statement 24

C

CALL task option 32

CANCEL option (<ACTION> statement) 30, 32

CANCEL task option

and no-display cards 47

card history 16

<CE> statement 35

<CENTER> statement 25

character set, for HDML 10, 24

<CHOICE> statement 33

choice cards 33

choice items

tasks in 16

CLEAR option (<ACTION> statement) 31

columns, formatting text in 25

comments, in HDML code 37

D

decks

defining in HDML 37

DEFAULT option (<CHOICE> statement) 34

DEFAULT option (<ENTRY> statement) 45

DEST option (<ACTION> statement) 30

DEST task option 40

destinations 40

examples 41

syntax 40

<DISPLAY> statement 41

display cards 41

E

EMPTYOK option (<ENTRY> statement) 45

<ENTRY> statement 43

entry cards 43

esc (variable option) 22

F

FORMAT option (<ENTRY> statement) 44

FRIEND option (<ACTION> statement) 31

G

GO task option 31
 and no-display cards 47
GOSUB task option 31
 and no-display cards 47

H, I, J

Handheld Device Markup Language (HDML)
 character set 10, 24
 formatted text 23
 syntax 9
<HDML> statement 37
history pruning 17
history stack 16
IDEFAULT option (<CHOICE> statement) 34
IKEY option (<CHOICE> statement) 34
IMAGE option (<ACTION> statement) 31

K

KEY option (<CHOICE> statement) 34
KEY option (<ENTRY> statement) 45

L

LABEL option (<ACTION> statement) 29
<LINE> statement 24

M

MARKABLE option (<CHOICE> statement) 33
MARKABLE option (<DISPLAY> statement) 42
MARKABLE option (<ENTRY> statement) 43
MARKABLE option (<HDML> statement) 38
METHOD option (<CHOICE> statement) 34

N

NAME option (<CHOICE> statement) 33
NAME option (<DISPLAY> statement) 41
NAME option (<ENTRY> statement) 43
NAME option (statement) 46
NAME option (<NODISPLAY> statement) 47
NEXT option (<ACTION> statement) 30
NOECHO option (<ENTRY> statement) 45
noesc (variable option) 22
NOOP task option 32

NUMBER option (<ACTION> statement) 31

O

options in HDML statements 13

P, Q

PREV task option 32
 and no-display cards 47
PUBLIC option (<HDML> statement) 38

R

RECEIVE option (<ACTION> statement) 30
reserved characters
 displaying 24
 list of 24
RETURN task option 32
 and no-display cards 47
RETVALS option (<ACTION> statement) 30
<RIGHT> statement 25

S

scrolling
 horizontal 24
 vertical 25
SENDREFERER option (<ACTION> statement) 31
spaces, in displayed text 25
SRC option (statement) 46

T

tabs, in displayed text 25
TASK option (<ACTION> statement) 29
task types 31
tasks 31
 in choice items 16
text
 formatting 23
 wrapping 24
TITLE option (<CHOICE> statement) 33
TITLE option (<DISPLAY> statement) 41
TITLE option (<ENTRY> statement) 43
TTL option (<HDML> statement) 38
TYPE option (<ACTION> statement) 29

U, V

UP.Phone vii

V

- variables 19
 - escaping 22
 - naming 20
 - passing between activities 23
 - referencing 21
 - scope 20
 - setting 20
 - user input 21
- VARs option (<ACTION> statement) 30
- VERSION option (<HDML> statement) 38

W, X, Y, Z

- whitespace characters 10
- <WRAP> statement 24
- wrapping text 24

